

FODAS: A Novel Reinforcement Learning Approach for Efficient Task Scheduling in Fog Computing Network

Ganesan Nagabushnam¹, Yundo Choi¹, and Kyong Hoon Kim^{1*}

¹School of Computer Science and Engineering

Kyungpook National University

Daegu, South Korea

ndganesan@gmail.com, ydchoi97@knu.ac.kr, kyong.kim@knu.ac.kr

Abstract—In heterogeneous fog-cloud computing networks, efficiently scheduling aperiodic tasks is an NP-hard problem, particularly when aiming to minimize makespan, adhere to deadlines, and conserve energy. This paper introduces a novel scheduling algorithm, Fog-Optimized Deadline-Adaptive Scheduling (FODAS), which combines Earliest Deadline First (EDF) principles with Deep Multi-Agent Reinforcement Learning, incorporating Proximal Policy Optimization (PPO) and Recurrent Neural Networks (RNN). FODAS is specifically designed to manage aperiodic tasks in heterogeneous fog-cloud environments, prioritizing deadline adherence and energy efficiency. The proposed algorithm begins by collecting tasks into a global scheduling queue, and sorting them by their deadlines. It incorporates three homogeneous schedulers within a heterogeneous framework, ensuring tasks meet their deadlines and achieve notable energy savings. Key performance metrics such as deadline meeting rate, makespan, and energy savings are evaluated, comparing FODAS against single-agent reinforcement learning algorithms such as PPO and Asynchronous Advantage Actor-Critic (A3C). Our findings reveal that FODAS significantly improves the rate of meeting deadlines by up to 18% compared to the conventional algorithms. Additionally, it delivers substantial energy savings, with improvements of up to 80% in certain setups, and markedly decreases makespan, achieving reductions of up to 57.3% compared to traditional algorithms. The proposed algorithm also demonstrates exceptional operational efficiency, reducing the time required for scheduling tasks, particularly in high-density node networks. These results underscore the effectiveness of FODAS in managing complex task scheduling within fog-cloud computing environments.

Index Terms—Fog computing, Cloud Computing, Task scheduling, Multi-Agent Reinforcement Learning, Deadline, Heterogeneous, Aperiodic task, Proximal Policy Optimization, Recurrent Neural Network

I. INTRODUCTION

The rapid growth of the Internet of Things (IoT) has led to an enormous increase in data generation, requiring robust computational infrastructure for effective data analysis and action triggering in various applications [1]. Traditional cloud-centric IoT solutions struggle with network latency, particularly in time-sensitive domains like healthcare and traffic management [2]. Fog-cloud computing emerges as a solution, offering reduced latency and enhanced Quality of Service (QoS) through optimized resource utilization. A key component in achieving

this optimization is an efficient scheduling system within the fog environment, balancing computational tasks, and meeting the demands of various IoT applications effectively.

Efficient scheduling encounters numerous challenges, primarily due to the stochastic nature of the fog-cloud environment as illustrated in Fig.1.

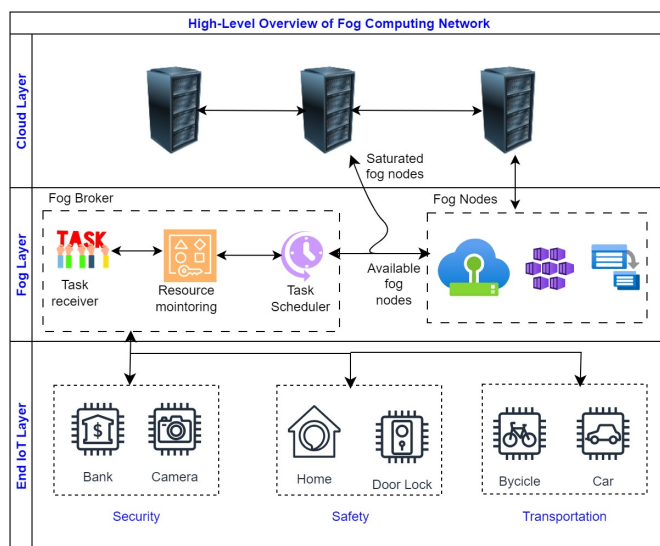


Fig. 1. fog-cloud architecture

Efficient scheduling encounters numerous challenges, primarily due to the stochastic nature of the fog-cloud environment. Factors contributing to its complexity include variability in task arrival rates, diverse task characteristics, heterogeneity of resources, and constraints related to energy and sustainability. Aperiodic task scheduling in such an environment is crucial for resource management. These tasks exhibit dynamic and stochastic characteristics, involving time-varying behaviors and unpredictable elements in task arrivals and resource demands. Reinforcement Learning (RL)-based scheduling approaches have been proposed for dynamic system optimization [3]. RL models are more accurate as they learn from real-world data and can identify intricate relationships among

dependent parameters. However, conventional RL techniques face scalability issues. Deep Learning-based RL techniques such as Deep Q Learning (DQN) aim to overcome these challenges, but studies have shown that these Q value-based techniques do not perform well in heterogeneous environments [4]. Moreover, they do not exploit task temporal patterns and network-node behavior to improve scheduling decisions.

Deep Multi-Agent Reinforcement Learning (DMARL) models [5] has emerged as a promising solution to address these challenges. The integration of deep learning (DL) with MARL allows multiple agents to collaboratively learn and adapt to specific aspects of the environment, resulting in a more comprehensive and flexible scheduling system. DL enhances MARL by enabling agents to maintain detailed state representations, including current energy levels, task loads, and node capabilities, leading to more informed and efficient scheduling decisions. DMARL supports scalability and effective resource utilization, with agents dynamically adjusting to environmental changes. This combination enables a recognition of complex patterns and dependencies in the data, improving task scheduling, reducing energy consumption, and enhancing overall system performance. The robust framework of DL and MARL offers an effective solution for task scheduling in fog-cloud networks.

Performance metrics such as deadline meeting rate, minimization of makespan, and energy consumption are fundamental challenges in task scheduling. Algorithms like Earliest-Deadline First (EDF) [6] prioritize tasks based on their deadlines, ensuring tasks with the earliest deadlines are scheduled first, enhancing the system's ability to meet deadlines and improving overall efficiency and user satisfaction.

A. Problem Formulation:

Considering the challenges in a fog-cloud environment, particularly for aperiodic tasks, we aim to minimize the makespan of these tasks while ensuring deadline adherence and energy efficiency. This approach is structured as an optimization task, represented mathematically. Table I lists the symbols used.

$$\text{Minimize } Z = \max_{j \in F} \left(\sum_{i \in T} x_{ij} \cdot e_{ij} \right) \quad (1)$$

subject to:

$$\forall i \in T, \sum_{j \in F} x_{ij} = 1 \quad (2)$$

$$\forall i \in T, \sum_{j \in F} x_{ij} \cdot e_{ij} \leq d_i \quad (3)$$

$$\sum_{i \in T} \sum_{j \in F} x_{ij} \cdot c_{ij} \leq T_{EC} \quad (4)$$

$$x_{ij} \in \{0, 1\},$$

where $\forall i \in T, \forall j \in F$

This paper proposes a novel DMARL-based scheduling algorithm—Fog-Optimized Deadline-Adaptive Scheduling (FODAS)—that integrates Multi-Agent Reinforcement Learning (MARL) with Proximal Policy Optimization (PPO) and

TABLE I
NOTATION TABLE

Symbol	Description
T	Set of aperiodic tasks, $T = \{t_1, t_2, \dots, t_n\}$
F	Set of heterogeneous fog nodes, $F = \{f_1, \dots, f_m\}$
Q	Global scheduler queue
S	Set of schedulers with PPO & RNN, $S = \{s_1, s_2, s_3\}$
e_{ij}	Execution time of task t_i on fog node f_j
c_{ij}	Energy consumption of task t_i on fog node f_j
d_i	Deadline for task t_i
x_{ij}	Binary variable: 1 if t_i is assigned to f_j , else 0
$R(s_j, t_i)$	Reward for scheduler s_j for scheduling t_i
$P(s_j, t_i)$	Penalty for scheduler s_j if t_i misses its deadline
T_{EC}	Total energy consumption for fog nodes
T_k^d	Tasks deadline
T_K^a	Tasks Arrival time
T_K^c	Tasks computation time

Recurrent Neural Networks (RNN) to enhance task scheduling in fog computing networks. The proposed FODAS model utilizes proximal policy optimization (PPO), a policy gradient algorithm recognized for its agility in adapting to task dynamics and resource diversity with balanced policy updates. Introduced by John Schulman and colleagues in 2017 [7], PPO distinguishes itself by its straightforward implementation, adaptability, and computational efficiency, excelling in managing vast state-action spaces. This capability ensures the proposed FODAS algorithm's swift adaptation to the unpredictable conditions characteristic of fog-cloud environments.

Along with this PPO algorithm, we also incorporate the RNN into the DMARL model. The RNN model enhances the model's capability to distinguish complex, nonlinear patterns from various input features effectively [8], and can also exploit the task temporal patterns for the huge number of hosts to make efficient scheduling decisions. As opposed to heuristic, meta-heuristic, and other RL-based methods, this proposed model can adjust to new application requirements as it improves continuously by tuning the hyper-parameters with new observations. The policy-based gradient method allows the proposed model to rapidly adapt the allocation policy in response to the dynamic task pattern, behavior of the host, and QoS demands more effectively. Also, to ensure the deadline-meeting rate of the aperiodic tasks received in the heterogeneous fog-cloud network, the EDF algorithm sorts the receiving tasks based on the deadline from the global scheduler queue. In this study, we perform the experiment using the Coupled Simulation and Container Orchestration (COSCO) Framework [9]. The proposed FODAS algorithm demonstrates superior performance across all the crucial metrics such as deadline adherence, makespan, and energy savings against traditional single-agent RL algorithms such as PPO and A3C. In summary, the main contributions of the paper are as follows:

- Proposing FODAS, a novel EDF based scheduling algorithm with PPO and RNN in a DMARL model for aperiodic task scheduling in a heterogeneous fog-cloud environment.
- Demonstrating how exploiting the task temporal patterns with RNN helps in an effective scheduling in a heterogeneous fog-cloud network.

The remainder of this paper is organized as follows: Section II delves into relevant prior research in this field, providing a foundation and context for this work. Section III details the proposed FODAS algorithm, developed as a DMARL framework with PPO and RNN, emphasizing how this algorithm achieves efficient scheduling of aperiodic tasks in a heterogeneous fog-cloud environment. Section IV evaluates the performance of the proposed FODAS algorithm, showcasing its effectiveness through various metrics. Finally, Section V concludes the paper, summarizing the findings and contributions to the field.

II. RELATED WORKS

In recent years, several researchers have developed many heuristic-based methods for task scheduling tasks in various distributed environments. These approaches primarily focus on optimizing specific parameters tailored to certain applications. While heuristics are generally effective for standard cases, they often fall short in adapting to dynamic environmental changes. In contrast, other learning models offer the advantage of continual improvement by adjusting their parameters in response to new data. This approach is evident in recent studies that explore predictive optimizations using Machine Learning (ML) and Deep Learning (DL) techniques for enhancing Resource Management Systems (RMS). Techniques like Deep Neural Networks (DNN) and Deep Reinforcement Learning (DRL) have been particularly prominent in this area. A significant focus of studies has been on energy optimization. For instance, Maharana et al. [10] proposed an energy-efficiency algorithm for aperiodic task scheduling in a cloud computing environment. Similarly, El Ghor and Aggoune [11] developed an energy-efficient algorithm as a variant of the EDF algorithm named ES-DVFS (Energy Saving-Dynamic Voltage and Frequency Scaling) for aperiodic task scheduling in Real-time Embedded systems. In line with these developments, a novel approach [12] advances this field by integrating the PPO algorithm to optimize energy use in data centers. Additionally, several works have explored RL-based algorithms for energy-efficient scheduling in distributed systems. For example, Cheng et al. [13] investigated DRL-based strategies for resource provisioning and task scheduling in cloud services using a Q-learning-based approach aimed at reducing electricity costs and task rejection rates. Li et al. [14] also explored RMS optimization using Double Deep Q Networks (DDQN), applying DRL to schedule jobs across multiple resources and achieving significant gains compared to conventional heuristic methods.

However, these Q-learning-based models often struggle to adapt quickly in stochastic environments. Therefore, we aim

to explore the use of Multi-Agent Reinforcement Learning (MARL) as it is well-suited for dynamic environments. For example, a study [15] presented a task allocation approach in dynamic environments using MARL, combining centralized training with decentralized execution. Researchers in [16] explored Multi-Agent Proximal Policy Optimization (MA-PPO) for network task scheduling. Their study highlights the increasing popularity of advanced reinforcement learning approaches, such as MA-PPO, for better task scheduling in complex network environments. MARL has been employed to address uncertainties and optimize task scheduling in dynamic environments. Furthermore, some studies [5] have combined Deep Learning with MARL, known as DMARL, which can be used for more effective task scheduling as shown in this study.

Given the limitations of heuristic and RL-based methods in dynamic, stochastic environments with varying task arrival rates, we propose a novel algorithm for effective aperiodic task scheduling. It is developed as an EDF-based DMARL that utilizes a policy gradient-based RL algorithm (PPO) with RNN for aperiodic task scheduling in a heterogeneous fog-cloud environment. This scheduling algorithm aims to dynamically optimize schedules for aperiodic tasks effectively.

III. PROPOSED ALGORITHM

In this section, following an overview of the system model, we present the novel FODAS scheduling algorithm designed for aperiodic task scheduling in a heterogeneous fog-cloud environment. The proposed algorithm is implemented within the heterogeneous system model, as detailed below:

A. System Model

This model is designed for scheduling aperiodic tasks in a heterogeneous fog-cloud environment addressing the stochastic nature of IoT applications. The system model includes::

- 1) **IoT Layer:** This layer forms the interface with users, where diverse requests are generated from a range of devices and applications. These requests are then transmitted to the fog broker via gateways, emphasizing the need for efficient task handling from the IoT layer.
- 2) **Fog Layer:** The main component of this layer is the fog broker, which acts as the controller of this layer, the fog broker comprises three primary components:
 - **Task Receiver:** This unit is responsible for the initial reception and decomposition of requests into tasks, with a focus on estimating task-specific details such as a number of instructions and deadlines.
 - **Resource Monitoring:** It continually assesses the fog nodes, informing the task scheduler about the current status and capabilities of these resources.
 - **Task Scheduler:** It is the key decision-maker, utilizing data from other components to optimally schedule tasks. Here, the proposed FODAS algorithm plays a crucial role, in ensuring efficient task scheduling within this heterogeneous fog-cloud environment.
- 3) **Cloud Layer:** This layer offers additional computational and storage resources, accommodating tasks that surpass

the fog node's capacities or require more complex processing.

In this system model, we implement the proposed FODAS algorithm as defined below:

Algorithm 1 Fog-Optimized Deadline-Adaptive Scheduling Algorithm

Require: T, F, S

Initialize task set $T \leftarrow \emptyset$

Initialize global scheduler queue $Q \leftarrow \emptyset$

for each new task t_i from IoT devices **do**

$T \leftarrow T \cup \{t_i\}$

$Q \leftarrow Q \cup \{t_i\}$

end for

$Q \leftarrow \text{EDF_sort}(Q, D)$ {Sort tasks using EDF}

for each scheduler $s_i \in S$ **do**

if s_i is available and Q is not empty **then**

Select task t_j from Q

Assign t_j to an available fog node n_k

if deadline $D(t_j)$ is met **then**

Assign reward $R(s_i, t_j)$

else

Assign penalty $P(s_i, t_j)$

end if

end if

end for

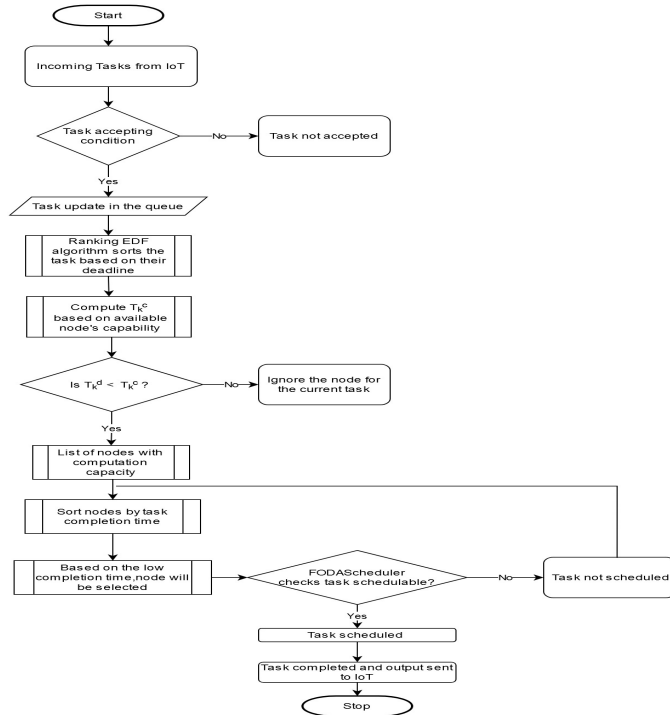


Fig. 2. Fog-Optimized Deadline-Adaptive Scheduling Algorithm flow chart

The overall flow of this algorithm is shown in Fig.2. The proposed Fog-Optimized Deadline-Adaptive Scheduling (FODAS) algorithm aims to efficiently manage aperiodic tasks in

a heterogeneous fog-cloud network. This detailed explanation outlines the sequential flow of the algorithm, emphasizing key decision points and processes involved.

- **Task reception and Initial Processing:** The process begins with the initiation of the scheduling algorithm. Tasks generated by IoT devices are received and enter the scheduling system. The first decision point involves checking whether the incoming task meets the predefined acceptance criteria. This acceptance criteria are mathematically defined as $T^d \geq T^a$, where T^d : deadline of the task and T^a : Task's arrival time.
- **Queue Management and Sorting:** Accepted tasks are added to the scheduling queue for further processing. The EDF algorithm is then employed to sort these tasks based on their deadlines, prioritizing tasks with the earliest deadlines. This ensures that tasks that have shorter deadlines are handled first.
- **Computation of completion time:** The next step involves calculating the completion time T_k^c for each task based on the capabilities of the available nodes. This calculation helps in assessing whether a task can be completed before its deadline.
- **Deadline Verification:** A critical decision point checks if the task's deadline T_k^d is less than the computed completion time T_k^c . If the deadline is less than the completion time, the task is considered for scheduling. If not, the node is ignored for the current task. This step ensures that only tasks that can be completed on time are considered for scheduling.
- **Node Selection:** Nodes with sufficient computational capacity to handle the task are listed. These nodes are then sorted based on the estimated completion time for the task. The node with the lowest estimated completion time is selected for task scheduling. This ensures that tasks are assigned to the most efficient nodes, optimizing resource utilization.
- **FODAScheduler check:** The FODAScheduler then evaluates whether the task can be scheduled on the selected node. If the task is schedulable, it proceeds to the next step. If not, the task remains unscheduled. Additionally, it updates the status of existing scheduled tasks.
- **Task Execution and Completion:** Once the task is deemed schedulable, it is assigned to the selected node. After the task is completed, the results are sent back to the IoT device. This final step concludes the scheduling process.

The following subsection explains the structure of the FODAScheduler:

B. FODAScheduler

The FODAScheduler, an integral component of the proposed DMARL scheduler, consists of multiple agents equipped with Proximal Policy Optimization (PPO) and Recurrent Neural Networks (RNN). This combination introduces a sophisticated framework for collaborative decision-making in multi-agent environments, enhancing the scheduler's ability to manage tasks efficiently in a heterogeneous fog-cloud network.

1) *Architectural Integration of PPO and RNN*: To provide a clear understanding of the integration, we detail the input and output processes of the RNN and PPO as follows:

Recurrent Neural Networks (RNN) Integration:

- **Input:** The RNN processes sequential data related to task arrivals, resource usage, and historical task completion times. Specifically, it takes a sequence of state observations $s_1, s_2, \dots, s_t$ as input, where each state s_t includes features such as current task size, energy levels, and node capabilities.
- **Output:** The RNN outputs a hidden state h_t that encapsulates the temporal dependencies and predictions about future states. This hidden state is then used as part of the input for the PPO policy network to make more informed decisions.

Policy Network Implementation for PPO:

- **Network Architecture:** The PPO policy network comprises an input layer that receives the current state (augmented by the RNN output), several hidden layers with non-linear activation functions ReLU, and an output layer that provides a probability distribution over possible actions. In this architecture, the state and action spaces are defined as :

$$s_t = (L_t, E_t, C, H_t, N_t) \quad (5)$$

and

$$a_t = (a_{assign}, a_{load}, a_{time}) \quad (6)$$

where L_t : current task loads on each node, E_t : Energy levels of each node, C : capabilities of each node (e.g., CPU), H_t : historical data on task completion times. N_t : Network conditions (e.g., latency), a_{assign} : Assigning tasks to specific fog nodes, a_{time} : Deciding the timing of task execution, a_{load} : Managing load distribution across nodes.

- **Value Function and Policy Function:** The Value function $V(s)$ represents the expected return (cumulative future reward) when starting from state s and following the current policy. it is used to evaluate the quality of different states.

$$V(s) = E \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right] \quad (7)$$

- **Policy Function:** The policy function $\pi(a|s)$ represent the probability of taking action a given state s . it define the agent's behaviour by mapping states to actions.

$$\pi(a|s) = P(A_t = a | S_t = s) \quad (8)$$

- **Action Selection:** Based on the output probability distribution, the policy network selects actions such as task assignment to specific fog nodes, timing of task execution, and load distribution management.

2) *Proximal Policy Optimization (PPO) Principle*:: PPO is a policy gradient method used to update the policy in a stable and efficient manner. The key principle of PPO is to limit the

extent of policy updates by introducing a clipping mechanism that prevents large updates that could destabilize the learning process.

- **Probability Ratio:** The probability ratio $r_t(\theta)$ between the new and old policies is calculated as:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \quad (9)$$

- **Clipping Objective function:** The clipped objective function ensures that the policy updates are not too large:

$$L_{CLIP}(\theta) = E_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right] \quad (10)$$

where $r_t(\theta_i)$: Represents the probability ratio for agent i . A^t : Denotes the advantage function at time step t .

Modified PPO objective function: The incorporation of Multi-Agent PPO with RNNs involves modifying the standard PPO objective to suit a multi-agent setting. The modified LCLIP objective function for multi-agent PPO, denoted as $L_{CLIP}(\theta_1, \theta_2, \dots, \theta_n)$ governs the optimization process for N agents. This objective function minimizes the weighted sum of individual agent rewards, controls update magnitudes through clipping, and considers the sum of advantages.

$$L_{CLIP_global}(\theta_1, \theta_2, \dots, \theta_N) = \min \left(\sum_{i=1}^N w_i r_{t,i}(\theta_i) A_{t,i}, \text{clip} \left(\sum_{i=1}^N w_i r_{t,i}(\theta_i), 1 - \epsilon, 1 + \epsilon \right) \sum_{i=1}^N A_{t,i} \right) \quad (11)$$

where weight w_i symbolizes the importance attributed to the contribution of each agent towards the overall goal, allowing for distinction in responsibilities or influence on the overall benefit.

To showcase this model in a consolidated and improved way, we combine the previous steps into one refined equation.

$$L_{MARL-PPO-RNN}(\theta_1, \theta_2, \dots, \theta_N) = \min \left(\sum_{i=1}^N w_i \frac{\pi_{\theta_{old,i}}(a_t|h_{t-1,i})}{\pi_{\theta_i}(a_t|h_{t-1,i})} (A_{t,i}), \text{clip} \left(\sum_{i=1}^N w_i \frac{\pi_{\theta_{old,i}}(a_t|h_{t-1,i})}{\pi_{\theta_i}(a_t|h_{t-1,i})}, 1 - \epsilon, 1 + \epsilon \right) \sum_{i=1}^N (A_{t,i}) \right) \quad (12)$$

Global Reward Advantage Function:

The advantage of each agent is determined using the global reward and estimated value function.

$$A_{t,i} = R_{global}(t) + \gamma V(s_{t+1}) - V(s_t) \quad (13)$$

The estimated value function of state s_t is denoted by $V(s_t)$, with γ representing the discount factor for future rewards.

The global reward aggregation is given by equation (14)

$$R_{global}(t) = \sum_{i=1}^N R_i(s_t, a_t) \quad (14)$$

This global reward sums the individual negative impacts (reward penalties) related to makespan and energy, encouraging agents to collectively find the most efficient solutions.

Probability Ratio with RNN Results: To integrate the decision-making procedure impacted by RNNs, the probability ratio is established in consideration of the RNN's ability to capture sequential dependencies.

$$r_{t,i}(\theta_i) = \frac{\pi_{\theta_{old,i}}(a_t|h_{t-1,i})}{\pi_{\theta_i}(a_t|h_{t-1,i})} \quad (15)$$

$h_{(t-1,i)}$ represents the RNN's hidden state for agent i at time $t-1$, guaranteeing that actions are influenced by previous observations and choices.

The integration of RNN and PPO within each scheduler agent significantly enhances system efficiency and responsiveness. RNNs excel at understanding and predicting temporal task patterns, essential for the dynamic nature of task arrivals in fog-cloud environments. This capability allows scheduler agents to make proactive, informed decisions, optimizing resource allocation and scheduling strategies. The predictive power of RNNs ensures efficient task distribution, balancing workload, and improving overall system performance. Coupled with a reward and penalty mechanism, the proposed algorithm encourages timely task completion and energy-efficient node utilization. Consequently, the proposed FODAS algorithm leverages the strengths of PPO and RNN, combined with the EDF algorithm, to effectively schedule aperiodic tasks, minimize makespan, adhere to deadlines, and enhance energy efficiency, thereby fostering a more robust and responsive IoT infrastructure.

IV. RESULTS AND DISCUSSION

In this study, we evaluated the proposed Fog-Optimized Deadline-Adaptive Scheduling (FODAS) algorithm using a system setup comprising heterogeneous Raspberry Pi devices as fog nodes, each differing in computational capacity. Tasks ranging from 500 to 5000 instructions, each with specific deadlines, were dynamically generated and processed within this framework. And, the power of the active fog nodes ranges from 90-180 Joules, and the task deadline ranges from 2000-4000 ms. The COSCO simulator was employed to generate synthetic data that closely replicates real-world IoT applications, providing a robust platform for evaluating the proposed algorithm. We conducted a comprehensive analysis to quantitatively assess the performance of the proposed FODAS algorithm against traditional single-agent RL algorithms, such as PPO and A3C, across a spectrum of fog computing environments. The COSCO simulator facilitated rigorous assessments through experiments designed to gauge the model's efficacy in managing varying loads of aperiodic tasks (100, 200, and 500) distributed across different node configurations (10, 30, 50, 70, and 90 fog nodes). The best performance was observed while using 3 agents in our proposed scheduler during the experimental study with 2 -7 agents. The evaluation metrics included deadline-meeting rate, energy savings, makespan,

and scheduler time, serving as benchmarks for performance comparison.

Hyperparameters for Learning To provide a detailed understanding of our implementation, the following hyperparameters were used:

- **Learning Rate:** The PPO and RNN models are trained using a learning rate of 0.0003, ensuring gradual and stable updates.
- **Replay Buffer:** A replay buffer of size 10,000 is maintained to store past experiences, enabling the agents to learn from historical data.
- **Number of Epochs:** The models undergo training over 20 epochs, iterating through the replay buffer to optimize the policy and value networks.
- **Discount Factor (γ):** A discount factor of 0.99 is used to prioritize immediate rewards while still considering future benefits.
- **Activation function:** ReLU (Rectified Linear Unit)

Deadline-Meeting Rate: The proposed FODAS algorithm significantly outperforms PPO and A3C in meeting deadlines, especially in smaller node configurations, as shown in Figures 3-5. For instance, with 100 tasks in a 10-node setup, the deadline-meeting rate improved by 14% over PPO and 18% over A3C. This advantage persists across all configurations, though gains decrease slightly in larger setups, indicating room for further optimization in dense networks.

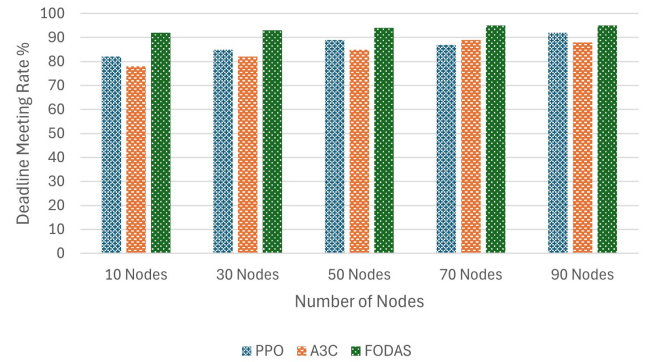


Fig. 3. Deadline Meeting Rate % for 100 Task

Energy Savings: Remarkable energy efficiency is observed with the proposed FODAS algorithm, as shown in Table 2. It doubled the energy savings in 10-node systems and improved by 78% over A3C in 90-node setups for 100 tasks. The benefit is pronounced in scenarios with up to 200 tasks but begins to diminish in 500-task configurations, suggesting scalability challenges in larger operations.

Makespan: The proposed FODAS algorithm exhibits substantial reductions in makespan across task distributions, achieving a 57.3% reduction compared to PPO in a 70-node configuration for 500 tasks, as illustrated in Figures 6-8. The relative advantage lessens with an increased number of nodes, pointing to potential enhancements needed for broader scalability.

TABLE II
ENERGY SAVINGS FOR DIFFERENT TASK LOADS AND NODE COUNTS

Node Count	100 Tasks			200 Tasks			500 Tasks		
	PPO	A3C	FODAS	PPO	A3C	FODAS	PPO	A3C	FODAS
10 Nodes	9.16	8.13	18.9	9.0	8.0	15	7.0	4.0	15.0
30 Nodes	12.09	10.4	29.9	10.0	9.0	18.0	9.0	6.0	13.2
50 Nodes	15.09	12.3	32.7	12.0	3.0	17.0	12.0	7.0	14.0
70 Nodes	13.1	20.0	34.6	24.0	12.0	16.6	8.0	10.0	10.0
90 Nodes	20.0	16.0	35.8	24.0	15.0	26.7	17.0	9.0	8.0

TABLE III
SCHEDULER TIME FOR DIFFERENT TASK LOADS AND NODE COUNTS

Node Count	100 Tasks			200 Tasks			500 Tasks		
	PPO	A3C	FODAS	PPO	A3C	FODAS	PPO	A3C	FODAS
10 Nodes	12.1	16.8	4.2	26.6	22.2	9.2	25.0	14.0	12.0
30 Nodes	17.0	19.2	6.2	32.2	32.1	7.2	29.0	29.0	13.2
50 Nodes	18.0	20.1	7.0	40.0	27.0	9.5	14.0	12.0	10.2
70 Nodes	20.0	20.0	7.4	31.0	32.0	10.4	8.7	12.0	9.8
90 Nodes	32.0	22.0	8.1	18.0	32.0	8.9	18.0	12.0	7.7

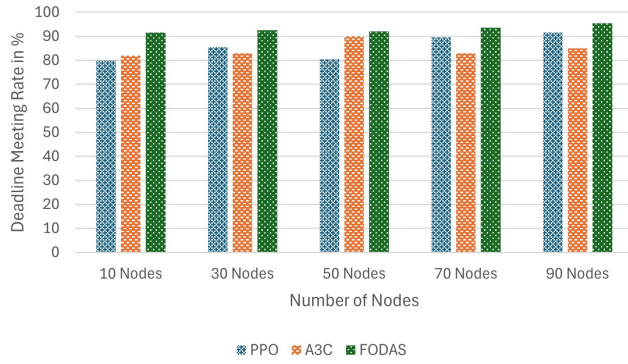


Fig. 4. Deadline Meeting Rate % for 200 Task

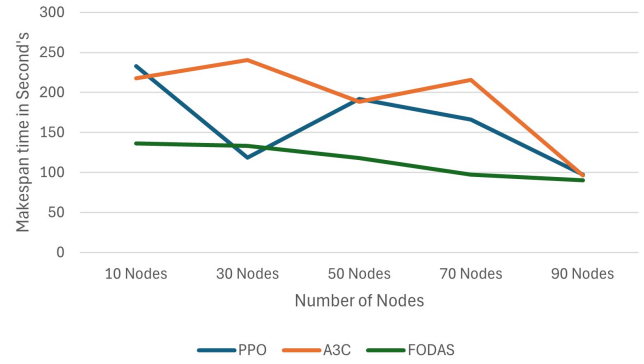


Fig. 6. Makespan in Seconds for 100 Task

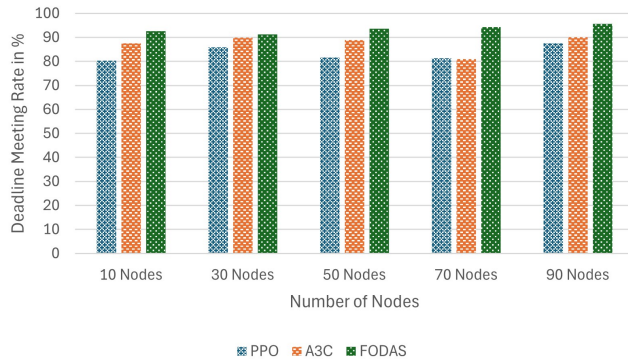


Fig. 5. Deadline Meeting Rate % for 500 Task

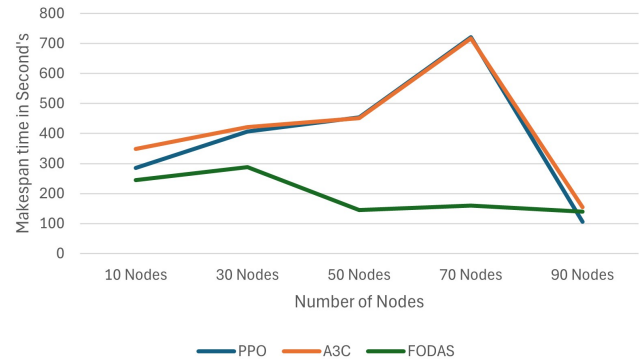


Fig. 7. Makespan in Seconds for 200 Task

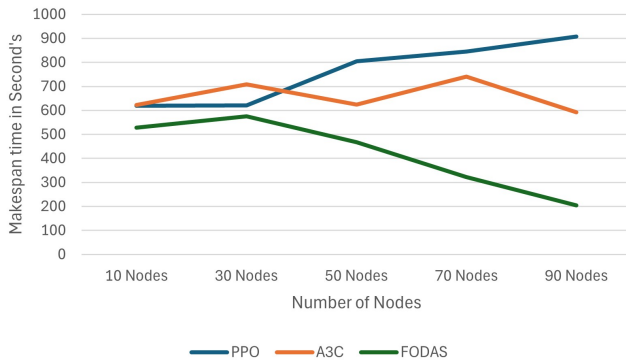


Fig. 8. Makespan in Seconds for 500 Task

Scheduler Time: The computational efficiency of the proposed FODAS scheduler is shown in Table 3. It considerably reduces scheduler time in a 10-node setup for 100 tasks, cutting it by 65.06% and 74.94% compared to PPO and A3C, respectively. While the algorithm maintains superior performance in denser networks, the rate of improvement declines as network size grows, indicating areas for future refinement.

Overall, the proposed FODAS algorithm demonstrates substantial improvements in scheduling aperiodic tasks within fog computing environments. It significantly enhances deadline adherence, energy savings, and operational efficiency, as evidenced by reduced makespan and scheduler times. These results highlight its potential to optimize real-time application performance and promote sustainable computing practices.

Despite the promising results, this study has certain limitations. The performance gains observed with FODAS decrease as the network size increases, indicating scalability challenges. The current model may not fully capitalize on the benefits of multi-agent reinforcement learning in extremely large or complex fog computing environments. Additionally, the study relies on synthetic data generated by the COSCO simulator, which, while robust, may not capture all nuances of real-world IoT applications.

V. CONCLUSION

The proposed Fog-Optimized Deadline-Adaptive Scheduling (FODAS) algorithm significantly outperforms traditional single-agent RL algorithms such as PPO and A3C in various performance metrics, including deadline-meeting rate, energy savings, makespan, and scheduler time. This algorithm shows substantial promise for enhancing task scheduling in heterogeneous fog-cloud environments, providing a framework that supports efficient resource utilization and real-time application performance. Future work should focus on addressing the scalability limitations identified, potentially through adaptive techniques or hierarchical scheduling models, to ensure consistent performance across larger and more complex fog computing networks. By overcoming these challenges, FODAS can

be positioned as a robust tool for optimizing fog computing systems.

ACKNOWLEDGMENT

This paper is part of the 4th phase BK21 project of the Ministry of Education and the National Research Foundation of Korea. This research was supported by (Intelligence Convergence Software Education Research Group, Department of Computer Science, Kyungpook National University) (41202420214871)

REFERENCES

- [1] R. Mahmud, S. N. Srirama, K. Ramamohanarao, and R. Buyya, "Quality of experience (qoe)-aware placement of applications in fog computing environments," *Journal of Parallel and Distributed Computing*, vol. 132, pp. 190–203, 2019.
- [2] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of things (iot): A vision, architectural elements, and future directions," *Future generation computer systems*, vol. 29, no. 7, pp. 1645–1660, 2013.
- [3] G. Fox, J. A. Glazier, J. Kadupitiya, V. Jadhao, M. Kim, J. Qiu, J. P. Sluka, E. Somogyi, M. Marathe, A. Adiga *et al.*, "Learning everywhere: Pervasive machine learning for effective high-performance computation," in *2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 2019, pp. 422–429.
- [4] Q. Zhang, M. Lin, L. T. Yang, Z. Chen, and P. Li, "Energy-efficient scheduling for real-time systems based on deep q-learning model," *IEEE transactions on sustainable computing*, vol. 4, no. 1, pp. 132–141, 2017.
- [5] A. Wong, T. Bäck, A. V. Kononova, and A. Plaat, "Deep multiagent reinforcement learning: Challenges and directions," *Artificial Intelligence Review*, pp. 1–34, 2022.
- [6] C.-g. Wu and L. Wang, "An estimation of distribution algorithm to optimize the utility of task scheduling under fog computing systems," *Fog Computing: Theory and Practice*, pp. 371–384, 2020.
- [7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [8] A. Sherstinsky, "Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network," *Physica D: Nonlinear Phenomena*, vol. 404, p. 132306, Mar. 2020. [Online]. Available: <http://dx.doi.org/10.1016/j.physd.2019.132306>
- [9] S. Tuli, S. R. Poojara, S. N. Srirama, G. Casale, and N. R. Jennings, "Cosco: Container orchestration using co-simulation and gradient based optimization for fog computing environments," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 1, pp. 101–116, 2021.
- [10] D. Maharana, B. Sahoo, and S. Sethi, "Energy-efficient real-time tasks scheduling in cloud data centers," *International Journal of Science Engineering and Advance Technology, IJSEAT*, vol. 4, no. 12, pp. 768–773, 2017.
- [11] H. El Ghor and E.-H. M. Aggoune, "Energy efficient scheduler of aperiodic jobs for real-time embedded systems," *International Journal of Automation and Computing*, vol. 17, pp. 733–743, 2020.
- [12] Y. Yang, C. He, B. Yin, Z. Wei, and B. Hong, "Cloud task scheduling based on proximal policy optimization algorithm for lowering energy consumption of data center," *KSII Transactions on Internet and Information Systems (TIIS)*, vol. 16, no. 6, pp. 1877–1891, 2022.
- [13] M. Cheng, J. Li, and S. Nazarian, "Drl-cloud: Deep reinforcement learning-based resource provisioning and task scheduling for cloud service providers," in *2018 23rd Asia and South Pacific design automation conference (ASP-DAC)*. IEEE, 2018, pp. 129–134.
- [14] F. Li and B. Hu, "Deepjps: Job scheduling based on deep reinforcement learning in cloud data center," in *Proceedings of the 4th International Conference on Big Data and Computing*, 2019, pp. 48–53.
- [15] D. B. Nouredine, A. Gharbi, and S. B. Ahmed, "Multi-agent deep reinforcement learning for task allocation in dynamic environment," in *ICSOFT*, 2017, pp. 17–26.
- [16] J. Zhao, K. Xie, H. Sun, C. Li, and Y. Zhang, "An ma-ppo based task scheduling method for computing first networks," in *Proceedings of the 2022 5th International Conference on Telecommunications and Communication Engineering*, 2022, pp. 260–265.